GatorTutors

SW Engineering CSC648-848 Fall 2025

Milestone 1

Team Lead:
William Wong (wwong23@sfsu.edu)

Frontend Team:

James Barr (Lead)

Randy Chen

Backend:
Uchenna Duru (Lead)

Ethan Niu

Github Maintainer:

Cyrus Molina-Perez

History Table:

Date Submitted	**10/17/25**
Date Revised	

1. Executive Summary

- **Motivation**

  At San Francisco State University, tutoring is vital but often limited by rigid schedules, course mismatches, and disconnected systems. The demographic of the student base is unlike most academic institutions in the sense that many students are commuters, working parents, and graduate researchers. These groups will struggle to find tutoring help that fits their class topics and time constraints. *GatorTutors* was created to solve this problem by building a unified, student-driven tutoring platform tailored for the SFSU community.

- **What GatorTutors Does**
  GatorTutors connects SFSU students and verified peer tutors through an intuitive web interface that simplifies booking, communication, and session tracking. Students can search for tutors by course number, subject, or availability, and instantly book either online or in-person sessions. Tutors can manage their schedules, upload notes, and review session histories. The platform automatically handles notifications and reminders, ensuring smooth coordination for everyone. All interactions happen securely within the site—no third-party emails or payments required—making the system both safe and campus-compliant.
- **Why GatorTutors is Unique to SFSU**
  Unlike generic tutoring services such as Kumon.com or Mathnasium, GatorTutors is designed *for and by* SFSU students. It integrates directly with the university's course catalog, allowing tutoring to be matched at the class-code level rather than by vague subject areas. This means a Computer Science student can find a peer tutor who has already taken "CSC 415" or "CSC 648" at SFSU and understands the exact curriculum and professor expectations. The platform also verifies users through SFSU email credentials, maintaining a safe, closed network exclusive to our campus community. What's not better than someone who's been through that exact class giving exclusive tips and tricks specific to the professor's curriculum.
- **Value and Impact**
  By bridging flexibility, familiarity, and trust, GatorTutors extends the reach of academic support far beyond traditional tutoring centers. It empowers students to learn from one another, reduces administrative overhead, and creates new opportunities for paid peer mentorship.
- **About Our Team**
  We are team 14, a team composed of five dedicated SFSU computer science students. The team is led by William Wong, with James Barr and Ethan Niu leading frontend development, Uchenna Duru and Randy Chen driving backend architecture, and Cyrus Molina-Perez managing version control.

2. Personae

Persona 1: Ryan

About:

- Graduate TA. Is a Master's student and working as a Teaching Assistant, offers tutoring as part of TA duties.
- Highly organized and driven academically, is enthused about teaching their subject matter.
- Knowledgeable in their field.

Pain Points:

- Finds logistics and repeating instructions annoying. Gets frustrated easily. Is less knowledgeable with technology, finds online sessions cumbersome.
- Prefers in-person tutoring.

Goals:

- Wants a way to easily organize recurring tutoring sessions, and wants to be easily able to share files/code snippets easily with students.
- Wants to be able to give students feedback on the platform directly.

Persona 2: Mary

About:

- Graduate student, working on a thesis research/capstone project. Lives off campus with their family (kids and spouse), and works a side job.
- Has a chaotic work-schedule due to juggling school, family, and work.
- Requires a tutor who specializes on certain topics, struggling with Biochemistry.
- Quite comfortable with technology.

Pain Points:

- Their chaotic schedule leads to only being available at late nights or at abnormal times. Is mostly only able to do online sessions.
- Chaotic schedule leads to complications setting up recurring sessions.

Goals:

- Wants a tutor to help catch up on lost time due to busy schedule.
- Be able to easily book recurring sessions with a tutor, wants flexible times, depending on their schedule.

- Wants a good way to keep track of sessions
- Prefers online sessions to fit into their schedule.

Persona 3: Alex

About:

- 18 year old, first year, full time computer science student with a packed school schedule.
- Lives in Sacramento and commutes daily by public transportation.
- Has no part time job currently.
- Doesn't have much experience with academic tutoring and online platforms.

Pain points:

- Only has basic computer and internet skills.
- Relies mostly on mobile phones and campus computers for digital access.
- Cant stay late on campus due to train schedules.
- Avoids overly technical or confusing interfaces.

Goals:

- Wants a mobile friendly interface to find and book tutors.
- Would want to see tutors that teach specific courses at SFSU. Filtering tutors based on course numbers.
- Possibility of remote sessions so they don't have to commute every time.

Persona 4: David

About:

- 22 years old, Senior in Computer Science at SFSU. Works as a tutor for specific Computer Science courses.
- Is passionate about programming and spends free time on personal coding projects.
- Known among peers for being tech-savvy and knowledgeable.
- Takes tutoring seriously and wants to give tutees a good experience, but expects them to be responsible and prepared.

Pain Points:

- Balances tutoring hours with senior-level courses and a part-time internship
- Has limited time for scheduling back-and-forth or tracking students manually
- Gets frustrated when students no-show or cancel last-minute without notice.

Goals:

- Wants a centralized platform where they can manage and accept bookings, set availability, and get reminders of appointments.
- Wants a built-in way to take notes on students and view them in future sessions.

3. High-Level Use Cases

- Use Case 1 – Filtering tutors by availability online or in-person

  Mary has an opening in her busy schedule at an abnormal time and is not able to make it to an in-person appointment. She goes to our site, and filters tutors by her time slot and whether they can do an online session. She reviews the tutors that are available according to her filters.

- Use Case 2 – Looking for course specific tutors

  Alex is struggling in his data structures class and wants to look for help via tutoring. Alex goes on our website with his phone and filters the tutors by ones that can help him with his data structures class. He looks for a timeslot that isn't too late so that he still has time to commute home afterwards.

- Use Case 3 – Managing and tracking tutoring sessions and notes

  David has finished a day of tutoring different students, and wants to keep track of future sessions he has and also write notes individually for each student. He goes to our website to see a list of all future appointments he has with each student, and he is also able to look at previous sessions that he has and is able to see the notes he took for each session.

- Use Case 4 – Booking and Receiving Session Reminders

  Mary has a busy schedule and often has trouble remembering the times of her tutoring appointments. As her appointments approach she receives reminder emails of her appointments a day before and an hour before her appointment. In the emails she is able to cancel or reschedule her appointments if she is no longer able to attend.

- Use Case 5 – Organizing Recurring In-Person Tutoring Sessions and Sharing Materials

  Ryan wants an easy way to set his tutoring hours and be able to manage sharing materials between students. He logs into our website to set his recurring availability weekly for in-person tutoring hours. After each sessions he is able to upload notes or code snippets which can be linked to the session(s) its associated to.

4. Main Data Items & Entities

- Key entities
    - Users
    - Students
    - Tutors
    - Courses
    - Enrollment
    - Transactions / Billing
    - Review
    - Availability

- Permissions (unregistered, registered, admin)
    - Unregistered
    - Registered
    - Tutors
    - Admin

- Any SFSU-specific entities
    - Professors

5. High-Level Functional Requirements

5.1 Unregistered User Requirements

1. The system shall allow unregistered users to browse available tutoring subjects and courses offered at SFSU.
2. The system shall allow unregistered users to view tutor profiles including their qualifications, ratings, and availability.
3. The system shall allow unregistered users to search for tutors by subject, course number, availability, and rating.
4. The system shall require users to create an account and verify SFSU affiliation before booking tutoring sessions.

5.2 Registered User Requirements

5. The system shall allow registered SFSU students to book tutoring sessions with available tutors for specific SFSU courses.
6. The system shall allow students to cancel or reschedule tutoring sessions at least 24 hours in advance.
7. The system shall allow students to view their upcoming and past tutoring sessions in a dashboard.
8. The system shall allow students to rate and review tutors after completing a session.
9. The system shall allow students to save favorite tutors for quick booking access.
10. The system shall send automated email notifications to students for session confirmations, reminders, and cancellations.
11. The system shall allow students to access session notes and materials uploaded by tutors after sessions are completed.
12. The system shall allow students to filter tutors by rating, availability, and specific SFSU course expertise.
13. The system shall allow registered SFSU tutors to create and manage their availability schedules for tutoring sessions.
14. The system shall allow tutors to specify which SFSU courses and subjects they are qualified to teach.
15. The system shall allow tutors to upload session notes, study materials, and resources for their students.

16. The system shall allow tutors to accept or decline booking requests from students.
17. The system shall allow tutors to view their session history and student feedback.

5.3 Admin Requirements

18. The system shall allow administrators to verify and approve tutor credentials and SFSU student enrollment status.
19. The system shall allow administrators to generate reports on tutoring session metrics, tutor performance, and platform usage.
20. The system shall allow administrators to manage user accounts including suspension or removal for policy violations.

5.4 SFSU-Specific Requirement(s)

21. The system shall integrate with SFSU's course catalog to automatically populate available courses and current semester offerings for tutor specialization selection

6. Non-Functional Requirements

1. Application shall be developed, tested and deployed using tools and servers approved by Class CTO and as agreed in M0

2. Application shall be optimized for standard desktop/laptop browsers e.g. must render correctly on the two latest versions of two major browsers

3. All or selected application functions shall be rendered well on mobile devices (no native app to be developed)

4. Posting of tutor information and messaging to tutors shall be limited only to SFSU students

5. Critical data shall be stored in the database on the team's deployment server.

6. No more than 50 concurrent users shall be accessing the application at any time

7. Privacy of users shall be protected

8. The language used shall be English (no localization needed)

9. Application shall be very easy to use and intuitive

10. Application shall follow established architecture patterns

11. Application code and its repository shall be easy to inspect and maintain

12. Google analytics shall be used

13. No e-mail clients shall be allowed. Interested users (clients) can only message service providers via in-site messaging. One round of messaging (from client to service provider) is enough for this application. No chat functions shall be developed or integrated

14. Pay functionality (e.g. paying for goods and services) shall not be implemented nor simulated in UI.

15. Site security: basic best practices shall be applied (as covered in the class) for main data items

16. Media formats shall be standard as used in the market today

17. Modern SE processes and tools shall be used as specified in the class, including collaborative and continuous SW development and GenAI tools

18. The application UI (WWW and mobile) shall prominently display the following exact text on all pages "SFSU Software Engineering Project CSC 648-848, Fall 2025. For Demonstration Only” at the top of the WWW page Nav bar. (Important so as to not confuse this with a real application).

7. Competitive Analysis

Inferior: ✖ Superior: ✔

Feature	Kumon	Mathnasium	Tutor.com	SFSU Tutoring	🟣 Our Platform
Scheduling	✖ Fixed sessions	✖ Fixed weekly schedule	✔ Calendar scheduling	✖ Manual form; no weekends	✔ Flexible online booking for tutors & students
Class specific tutoring	✖ Subject level only	✖ General skills only	✖ Subject based only	✖ Subject based only	✔ Course-code matching; peer tutoring for matching classes
One-on-one tutoring	✖ Groups only (4–6)	✖ Group-based	✔ Private sessions available	✖ 1/week limit, few STEM tutors	✔ 1:1 sessions anytime, any subject
Online learning	✖ In-person only	✖ Minimal tech	✔ White board, chat, file share	✔ Zoom sessions available	✔ Built-in Zoom-style links & instant messaging
University integration	✖ Franchise	✖ Franchise	✖ No SFSU integration	✔ Connected to departments	✔ Uses SFSU class codes, verified tutors, hybrid on/off-campus

Our SFSU Tutoring Platform directly targets the biggest weakness of existing tutoring services — a one-size-fits-all approach and an utter lack of university integration. While private competitors like Tutor.com focus on convenience, they lack campus alignment and class-level context. Kumon and Mathnasium excel at standardized learning but are totally irrelevant when it comes to higher education. SFSU's existing TASC offers strong departmental support but struggles to provide individualized scheduling, consistent STEM coverage, and any form of peer-to-peer collaboration, which in some cases can be more helpful than traditional generalized tutoring.

Our system blends the best of all worlds: real-time scheduling, class-specific matching, verified SFSU tutors, and built-in online tools for both one-on-one and group learning. By empowering students themselves to become peer tutors within their enrolled courses, our platform expands availability, reduces wait times, and enhances the sense of academic community — all while remaining tightly integrated with SFSU's systems.

8. High-Level System Architecture & Technologies

Main Software Components and Versions

- **Backend Runtime:** Node.js v20.11.1
- **Web Framework:** Express.js v4.19.2
- **Database:** MySQL 8.4.2
- **Operating System / Server Environment:** Ubuntu on AWS EC2
- **Version Control:** Git / GitHub

Deployment Cloud Service

- **Amazon Web Services (AWS)** — EC2 instance for server and database hosting

Front-End Frameworks / Technologies

- HTML / CSS / JavaScript

APIs

- Google Analytics

Supported Browsers

- **Google Chrome:** V114 - 141
- **Mozilla Firefox:** V113 - 140

9. Use of GenAI Tools (All, compiled by Leader)

The primary use of Generative AI tools occurred in Sections 1, 2, 3, 5, and 7, all during the ideation and inspiration stages of project development. AI tools were used solely to assist with brainstorming, formatting, and conceptual framing—no direct code, content, or design elements were copied without review or revision by team members.

Section 1 (Executive Summary):

AI was used to refine the project vision, clarify motivation, and structure the problem statement for clarity and conciseness.

Section 2 (Personae):

AI assisted in generating initial persona ideas, which were later refined to reflect real SFSU student demographics and tutoring use cases.

Section 3 (High-Level Use Cases):

AI supported the ideation process for realistic, scenario-based use cases. The final versions were reviewed and customized by the frontend team to ensure relevance to SFSU's academic environment.

Section 5 (Functional Requirements):

AI helped format and organize requirement statements according to standard software engineering conventions. All final requirements were written, verified, and validated by the backend team.

Section 7 (Competitive Analysis):

AI was used to brainstorm key comparison features and structure the competitive ranking table. Data and conclusions were finalized by the team after independent analysis of existing tutoring services.

Overall, AI served as a collaborative ideation partner to streamline brainstorming and improve document structure. Every section's final content was critically reviewed, rewritten, and validated by human team members to ensure accuracy, originality, and alignment with SFSU's academic standards.

10. Team & Roles

Name	Role	Email
William Wong	Team Lead	wwong23@sfsu.edu
Jamie (James) Barr	Frontend Lead	jbarr@sfsu.edu
Ethan Niu	Frontend	eniu1@sfsu.edu
Uchenna Duru	Backend Lead	uduru2@sfsu.edu
Randy Chen	Backend	rchen10@sfsu.edu
Cyrus Molina-Perez	Github Maintainer	cmolinaperez@sfsu.edu

11. Team Lead Checklist

Task	Status
So far all team members are fully engaged and attending team sessions when required	OK
Team found a time slot to meet outside of the class	DONE, every Tuesday
Team ready and able to use the chosen back and front end frameworks and those who need to learn are working on learning and practicing	DONE, using a tech stack from 317
Team reviewed class slides on requirements and use cases before drafting Milestone 1	DONE
Team reviewed non-functional requirements from “How to start...” document and developed Milestone 1 consistently	DONE
Team lead checked Milestone 1 document for quality, completeness, formatting and compliance with instructions before the submission	DONE
Team lead ensured that all team members read the final M1 and agree/understand it before submission	DONE
Team shared and discussed experience with GenAI tools among themselves	DONE
Github organized as discussed in class (e.g. master branch, development branch, folder for milestone documents etc.)	DONE